

# **Abenix**

# **The Edge Agent Guide**

*How AI agents in the cloud and on the plant floor cooperate over MQTT — for clients, plant operators, and application developers.*

---

An Abenix client-facing reference document.

# Contents

## **The Abenix Edge Agent Guide**

### **Part I — The big picture**

### **Part II — Agent fundamentals**

### **Part III — The MQTT bus, explained gently**

### **Part IV — How edge and cloud work together**

### **Part V — Real-life scenarios**

### **Part VI — How an application uses Abenix**

### **Part VII — Anatomy of one execution**

### **Part VIII — Security and governance**

### **FAQ**

### **Glossary**

# The Abenix Edge Agent Guide

---

A walkthrough for plant operators, application developers, and IT/OT leads. No prior knowledge of AI agents, MQTT, or Kubernetes is assumed.

---

## Who this is for

Three kinds of readers. Most chapters are useful for all three.

If you run operations or sponsor projects, you want to know what an Abenix edge agent is, why it exists, and what it actually does on the floor. Skip the YAML and the code samples; the diagrams and Part V are for you.

If you write software that integrates with Abenix, you want to know how to call the platform, how an agent is built, and how cloud and edge agents talk to each other. Read the whole thing; Part VI is your home.

If you handle plant IT or OT, you want to know what gets installed where, how the bytes move, and how the security model works. Parts III, IV, and VIII are written for you.

---

## What this guide covers

Abenix is a platform for building AI agents. An agent is a small program that reads some information, reasons about it, and takes an action. The platform lets you run an agent two ways: in the cloud, where it has access to large knowledge bases and a full set of integrations, or on the plant edge, where it sits next to the equipment and responds in milliseconds.

The same agent definition can run in either place. You author once and decide later where it executes.

The two halves talk through MQTT, a small message protocol that has been the standard for industrial telemetry for two decades. Cloud agents publish work to the edge over MQTT. Edge agents publish telemetry and escalations back. This guide explains how that all fits together.

---

# Part I — The big picture

---

## Three real problems

A vibration sensor on a wind turbine spikes for one second. The operator has ten seconds to decide whether this is a bearing about to fail, in which case the turbine should be shut down, or a wave hitting the platform, in which case the reading should be ignored. Sending the raw 50 kHz vibration stream to a cloud model is too slow and too expensive, and it stops working when the satellite link is flaky.

A pharma shipment of vaccines drops below 2 °C for nine minutes. The regulations say someone has to decide: was this a real loss? Was the cooling system at fault? Who pays? The decision needs the policy KB, the SOP knowledge graph, the prior-incident history, and human signoff. None of that fits on the truck.

A field technician at a remote substation needs an OEM-cited repair procedure for a fault code. She has spotty 4G. She speaks the symptom, uploads a damage photo, and wants the procedure, the safety gate, and a populated work order. The cloud may or may not be reachable.

These three situations all need reasoning, not just retrieval. They differ in where the reasoning belongs. The first one wants the model on the gateway. The second one wants it in the cloud with a human gate. The third one wants both: the device handles partial work, the cloud finishes the rest when the link comes back.

Abenix lets you write the reasoning once and pick the deployment shape later.

## Cloud agents and edge agents in one paragraph each

A cloud agent is the equivalent of a senior person at head office. They have access to the entire knowledge base, the email system, and the corporate finance tools. They take on slow, multi-step problems. They can call other agents. They can ask for human signoff before they act.

An edge agent is the equivalent of a field engineer with a toolbox at the site. They see the equipment directly. They make fast local decisions on a small set of rehearsed problems. They report back to head office. They escalate anything outside the playbook.

The two work together. The cloud agent decides what the edge agent should focus on. The edge agent escalates what it cannot handle. They use a common channel, which is MQTT.

## Why MQTT

MQTT is a publish-subscribe message protocol that has been around since 1999. It is small (a typical message header is a handful of bytes), it is bandwidth-efficient enough for satellite or cellular links, and it survives flaky networks because messages can be queued and replayed.

Many plants already run an MQTT broker. SCADA systems and historians often speak it natively. Using MQTT as the cloud-to-edge channel keeps Abenix out of the way of the plant's existing networking, which means no new firewall holes, no new vendor relationships, and no new protocol for the OT team to learn.

---

# Part II — Agent fundamentals

---

## What is an agent

An agent in Abenix is a small program made of three things.

A name and a description. For example, `iot-pump-edge-classifier`, with a description like "you receive a vibration window, you return a severity verdict."

A model and a system prompt. The model is the language model the agent uses (Claude, GPT, or Gemini). The system prompt is the standing instruction that shapes every conversation the agent has.

A list of tools. Tools are the agent's hands. They include things like `mqtt_publish` to send a message to a topic, `tsdb_query` to read time-series data, `connector_call` to talk to SAP or ServiceNow, and `code_executor` to run a snippet of Python in a sandbox.

When the agent runs, it gets some input (a JSON object, a file, a text prompt). It thinks about the input using the model. If it needs to act on the world, it calls a tool. The tool's output goes back into the conversation. The agent thinks again. The loop continues until the agent decides it is done, then returns a final structured result.

That loop is the only thing an agent ever does. Different problems need different combinations of tools, prompts, and models. The loop is the same.

## The tool families

Abenix ships about 100 tools out of the box. They group into nine families.

| Family           | Examples                                                                                                                  |
|------------------|---------------------------------------------------------------------------------------------------------------------------|
| Web              | <code>web_search</code> , <code>web_scrape</code> , <code>structured_extract</code>                                       |
| Knowledge        | <code>kb_search</code> , <code>kb_ingest</code> , <code>atlas_traverse</code> (graph walk)                                |
| Code             | <code>code_executor</code> for sandboxed Python, Node, Go, Rust, Java, or Ruby                                            |
| Data             | <code>tsdb_query</code> , <code>postgres</code> , <code>s3</code> , <code>csv_reader</code> , <code>parquet_reader</code> |
| Comms            | <code>slack</code> , <code>email_sender</code> , <code>webhook</code>                                                     |
| Productivity     | <code>linear</code> , <code>jira</code> , <code>notion</code> , <code>github</code>                                       |
| Vision and audio | <code>image_analyzer</code> , <code>audio_stt</code>                                                                      |
| Industrial       | <code>mqtt_publish</code> , <code>mqtt_subscribe</code> , <code>opcua_write</code> , <code>connector_call</code>          |
| Governance       | <code>approval_gate</code> , <code>windowed_state</code> , <code>subscribed_feed</code>                                   |

A platform admin curates which tools each agent can use. An agent that files work orders gets `connector_call` to ServiceNow. An agent that does sub-second alarm triage gets `mqtt_publish` and `windowed_state`.

## How an agent runs

```

Input
|
v
Read input + context
|
v
Reason with the model <--> LLM API
|
v
Wants a tool? ---- No ---> Return final answer
|
Yes
|
v
Call tool <--> Tool output
|
+-----> back to "Reason with the model"

```

Every agent in Abenix runs that loop. The differences between a cloud agent and an edge agent are limited to three things: which tools are allowed, where the agent's process lives, and which model it talks to.

## A cloud agent example: claim adjudication

A customer files a car insurance claim. The platform receives the FNOL (First Notice of Loss): a description, a damage photo, and the policy number. Six agents run in sequence as a cloud pipeline.

The intake agent parses the free text into structured fields. The policy match agent uses `kb_search` to find the clauses in the policy KB that govern this claim type. The damage assessment agent uses `image_analyzer` to estimate severity from the photo. The fraud screen agent uses `windowed_state` to check for repeat-claim patterns from the same customer. The valuation agent uses `connector_call` against a parts-pricing service. The decider agent puts everything together and proposes approve, partial, or deny with cited clauses.

The whole pipeline runs in the cloud because it needs the full policy KB, the multimodal vision model, and the parts-pricing connector. It is not fast (typically 8 to 15 seconds per claim) but it is auditable: every clause cited, every photo measurement logged, every connector call recorded.

## An edge agent example: pump vibration classifier

Same platform, different problem. A sensor on a wind-turbine pump publishes one vibration window per second to MQTT topic `pump/vibration.raw`. The verdict (low, medium, or high severity) needs to come back within a few tens of milliseconds, and it cannot depend on the satellite link being up.

The edge agent has a short system prompt, a small fast model, two tools, and a tight set of edge constraints.

```
name:      iot-pump-edge-classifier
model:     claude-haiku-4-5
tools:     code_executor, mqtt_publish
constraints:
  max_payload_bytes: 8192
  max_runtime_seconds: 5
  mqtt_publish:      [plant/pump/severity]
  mqtt_subscribe:    [plant/pump/vibration.raw]
```

This agent runs on a small pod sitting on the plant LAN, next to the SCADA system. The pod was helm-installed once, registered itself with the platform, and now hot-loads new versions of the agent whenever the platform pushes one over MQTT.

Both agents use the same SDK and the same YAML format. The difference is where they run.

---

# Part III — The MQTT bus, explained gently

---

## The post-office analogy

Imagine a town with a single post office. Anyone can drop a postcard addressed to a topic, like `weather/rain` or `mayor/announcements`. Anyone can subscribe to a topic, and the post office automatically forwards every postcard on that topic to them.

That is MQTT.

The post office is the broker. Abenix uses mosquitto, the most popular open-source broker. It runs in about 30 MB and ships with our docker-compose and our helm chart.

A postcard is a message. It is just bytes, usually JSON or plain text, typically a few hundred to a few thousand bytes long.

A mailbox is a topic. Topics are forward-slash-separated strings, like file paths: `pump/vibration.raw`, `cold-chain/truck-42/temperature`, `plant/floor-3/alarm/severity`.

## Topics and wildcards

Topics are written as paths. Subscribers can use wildcards.

The pattern `pump/vibration.raw` matches exactly that topic. The pattern `pump/+/temperature` matches one level (`pump/eastern-1/temperature`, `pump/eastern-2/temperature`, and so on). The pattern `cold-chain/#` matches everything under `cold-chain`.

A common topic shape is `<site>/<asset-type>/<asset-id>/<measurement>`, which gives you names like `plant-3/pump/p-12/vibration-rms`. Good topic naming is half the battle.

## Publish, subscribe, and the three QoS levels

There are two verbs in MQTT. Publish drops a message on a topic. Subscribe tells the broker which topics you care about and starts receiving forwarded messages. Both verbs take a quality-of-service level (QoS) that controls delivery guarantees.

| QoS | Meaning                                              | When to use                                                       |
|-----|------------------------------------------------------|-------------------------------------------------------------------|
| 0   | "fire and forget" - at most once delivery            | High-volume telemetry where loss is acceptable.                   |
| 1   | "at least once" - guaranteed but possibly duplicated | Most operational messages: alarms, deploy events, status updates. |
| 2   | "exactly once" - guaranteed and deduplicated         | Rare. Use only for critical settlement-style messages.            |

Abenix publishes bundle deliveries at QoS 1, since we want them to land. High-frequency telemetry usually goes at QoS 0. Command-and-control messages go at QoS 1.

## Retained messages

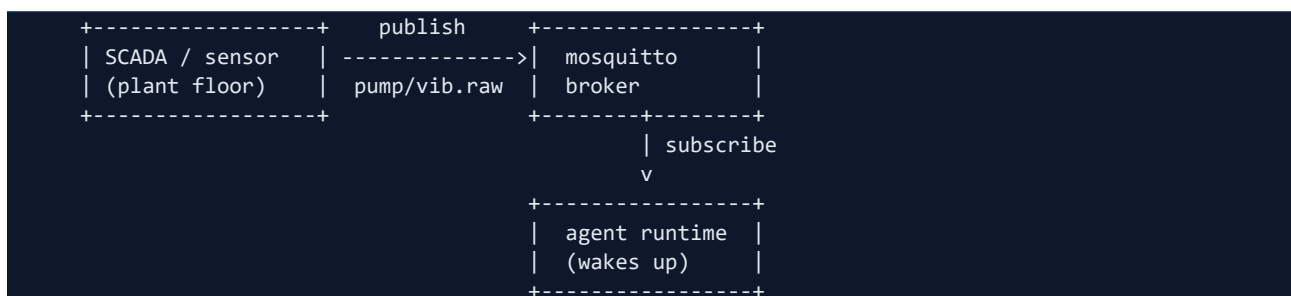
Normally, a subscriber that joins late does not see old messages: they were already forwarded. MQTT has a flag called [retained](#). When set on a publish, the broker keeps the latest message on that topic and replays it to every new subscriber.

This is a clean way to expose a "current state" topic. For example, a pump's current state can live on [plant-3/pump/p-12/state](#) as a retained message. Any new dashboard joining the broker sees the current state immediately, with no polling.

## The four ways Abenix uses MQTT

Four patterns. Once you understand these, the rest of the edge story falls out.

### Pattern 1: sensor to agent (telemetry trigger)



The sensor publishes telemetry on a topic. An agent's runtime is subscribed to that topic. Every new message wakes the agent and feeds the bytes in as input. No polling, no glue code.

### Pattern 2: agent to equipment (command-and-control)



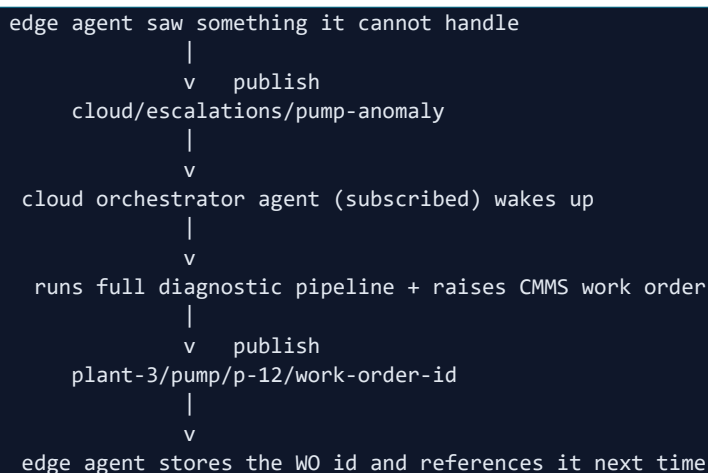
An agent decides on an action and publishes a command. The equipment (or a small bridge in front of it) is subscribed and acts on the command. The agent's `mqtt_publish` tool has a per-agent ACL that restricts which topics it is allowed to publish to.

### Pattern 3: platform to edge (signed bundle delivery)



The platform compiles an agent into a `.agent` bundle (a signed tarball), publishes the bytes on the gateway's deploy topic, and the runtime hot-loads it. This is how a cloud-authored agent gets onto the edge.

### Pattern 4: edge to cloud and back (handoffs)



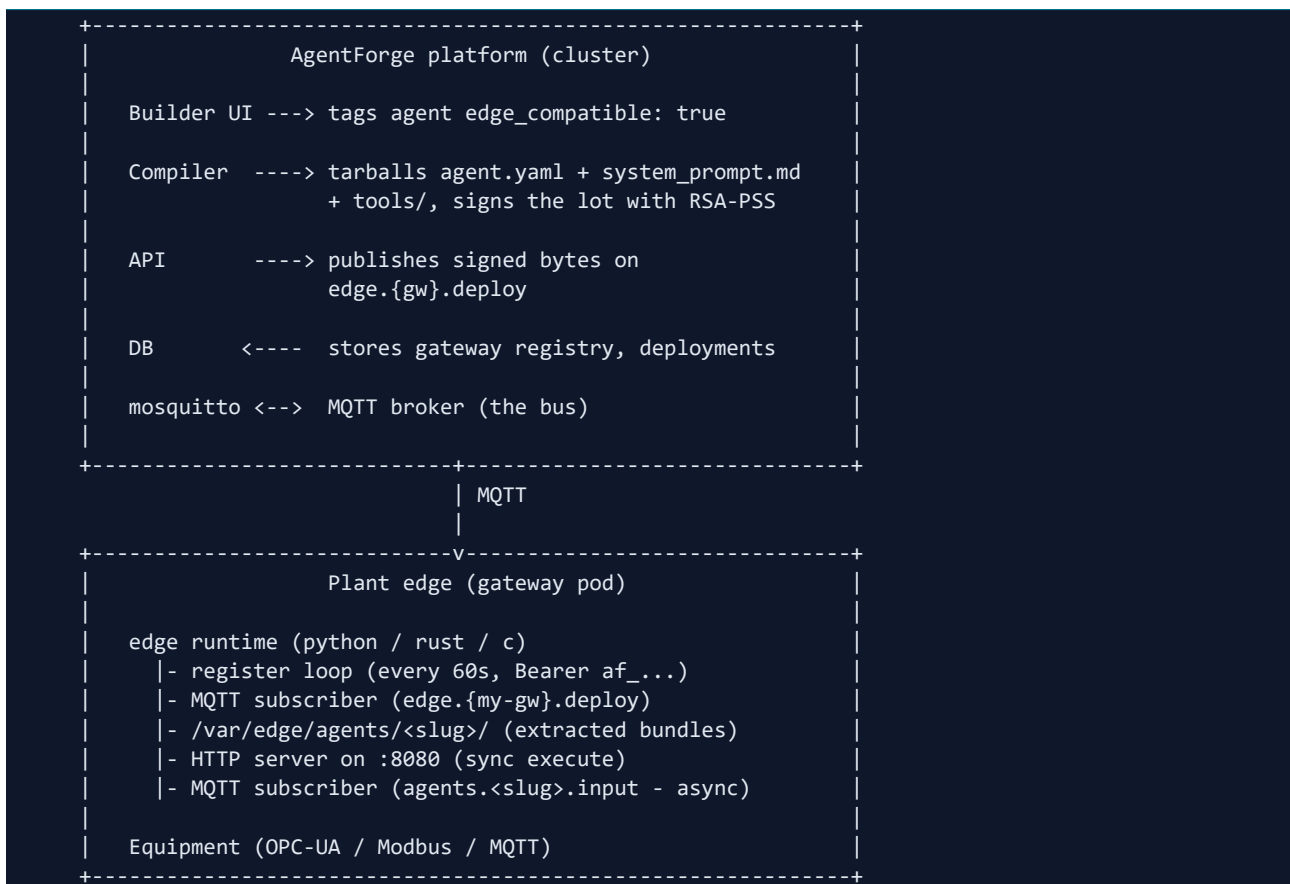
An edge agent handles common cases locally and hands the rest off to a cloud agent by publishing on an escalation topic. The cloud agent does the deep work and publishes its result back. This is the everyday cooperation pattern between cloud and edge.



# Part IV — How edge and cloud work together

This part walks the full lifecycle, from the moment a developer marks an agent as edge-eligible to the moment it is running on a turbine.

## The big picture



Now the steps.

## Step 1: authoring

A developer or a domain expert opens the Builder in the web UI. They configure the agent: name, slug, model, system prompt, the list of tools, and the edge-eligibility flag. When they tick "edge eligible", the Builder validates the configuration against the edge whitelist (only six tools are allowed) and requires the agent to declare its `edge_constraints`: max payload, max runtime, allowed MQTT topics.

The agent is now in the catalogue with status `draft`. It can be tested in the cloud first: run it through the SDK, run it from the Builder's test panel, run it against a fixture set. You debug in

the cloud, you ship to the edge.

## Step 2: provisioning a gateway

In parallel, the plant IT team prepares the gateway. Three steps.

First, mint a registration token. From the platform UI, go to **API Keys -> New key** and give it the scopes `agents:execute`, `edge:register`. The `af_` value is what the gateway will use to authenticate.

Second, pick a runtime variant. Python (~80 MB) is the reference variant and the easiest to extend. Rust (~25 MB) is a single static binary good for rugged industrial PCs. C (~12 MB) is a musl-linked binary for very constrained gateways. The [/edge](#) page shows three cards with copy-paste install commands.

Third, helm-install on the gateway:

```
helm install abenix-edge ./infra/helm/edge-runtime \
-n abenix-edge \
--set platform.url=https://abenix.your-corp.com \
--set platform.token=$EDGE_REGISTRATION_TOKEN \
--set gateway.id=plant-3-gw-1 \
--set mqtt_url=mqtt://mqtt.your-plant:1883 \
--set anthropic_api_key=$ANTHROPIC_API_KEY
```

The pod boots and within 60 seconds two things happen. The pod calls `POST /api/edge/gateways/register`, identifying itself with `gateway_id: plant-3-gw-1`. The pod subscribes to MQTT topic `edge.plant-3-gw-1.deploy`, which is its private deploy channel.

Refresh the platform's [/edge](#) page. The new gateway appears in the **Registered gateways** list with `last_seen_at` showing about 12 seconds ago.

## Step 3: compiling the bundle

The developer goes to [/edge](#), clicks the gateway card, clicks **Deploy agent**, and picks the agent. The platform does six things.

It validates the agent is still edge-eligible. It pulls the YAML, the system prompt, and any inline tools from the database. It builds the bundle as an uncompressed tar containing `agent.yaml`, `system_prompt.md`, optional `tools/*.py`, and (reserved for a future release) optional `model_weights/`. It signs the bundle with RSA-PSS over a SHA-256 digest. It publishes the bundle bytes to MQTT topic `edge.plant-3-gw-1.deploy` at QoS 1. It records the deployment in the database with the bundle digest.

If MQTT publish fails for any reason, the platform falls back to a direct HTTP POST against the gateway's `endpoint_url`. Either way, the bundle reaches the gateway.

## Step 4: hot-loading on the edge

The edge runtime, subscribed to `edge.plant-3-gw-1.deploy`, receives the bundle bytes.

It loads the platform's public key from `/etc/edge/signing_pub.pem` (mounted by helm), separates the signature from the rest of the tar, recomputes the digest, and verifies the signature. A mismatch means refuse to load and log the attempted tampering.

It re-runs the tool whitelist check. Belt and braces: even if the compiler already enforced it, the runtime checks again.

It extracts the tar to `/var/edge/agents/iot-pump-edge-classifier/`.

It registers the slug in its in-memory router. Now `GET /agents` lists the slug and the bundle digest. `POST /agents/iot-pump-edge-classifier/execute` is live.

If the manifest configured async invocation, the runtime also subscribes to `agents.iot-pump-edge-classifier.input`.

Total time from "click Deploy" to "agent is live" is typically under a second on a fast LAN. The runtime hot-loads: no pod restart, no downtime, no impact on other agents already deployed on the same gateway.

## Step 5: running the agent

There are two ways to invoke an edge agent.

The synchronous way is HTTP. A SCADA bridge or a small service on the plant LAN sends a POST:

```
POST http://abenix-edge.plant-3:8080/agents/iot-pump-edge-classifier/execute
Content-Type: application/json
{"samples": [0.012, -0.014, 0.011, ...], "sample_rate_hz": 4096}
```

The runtime executes the agent loop. The agent uses `code_executor` to run a Python snippet that computes RMS and the dominant FFT frequency, applies the rules in its system prompt, and returns a structured result:

```
{
  "slug": "iot-pump-edge-classifier",
  "duration_ms": 47,
  "result": {
    "severity": "high",
    "peak_hz": 102.3,
    "rms_g": 0.61
  }
}
```

If [severity](#) is high, the agent's last move is to publish to [plant-3/pump/p-12/severity](#). The plant's alarm desk dashboard, subscribed to that topic, lights up.

The asynchronous way is MQTT. Configure a trigger on the agent's manifest pointing at a topic pattern, redeploy, and the runtime subscribes. Every message on that topic feeds the agent automatically. The agent runs, publishes its verdict, and goes back to listening.

## Step 6: day-2 operations

The agent is running. Over time:

The platform monitors the gateway. Every 60 seconds the runtime re-registers. If it misses a check-in for more than five minutes, the dashboard flips the gateway to [stale](#) and an alert fires.

Updates ship over the same channel. The developer changes the prompt or swaps a model, clicks Deploy again, and the platform pushes a new bundle on the same topic. The runtime hot-loads it. No downtime.

The audit trail is unified. Every edge execution emits a structured log line that flows back to the platform's observability stack. Cloud and edge executions sit in the same [/executions](#) page, with the same Grafana dashboards and the same [/alerts](#) failure-code rollups.

That is the complete edge lifecycle.

---

# Part V — Real-life scenarios

Five end-to-end stories. Each one is built on patterns and components that ship in this repository.

## Scenario 1: wind turbine vibration monitoring

The plant is an offshore wind farm. Each turbine has accelerometers on the gearbox, bearings, and blade roots. Vibration data is published continuously to a local MQTT broker on the platform jack-up. The cloud link is a satellite uplink with hundreds of milliseconds of latency and frequent dropouts during storms.

The problem is that a bearing failure that goes unspotted for a day can grow into a gearbox replacement. Catching it at the early-warning stage is a much smaller intervention.

This scenario uses two agents that have been authored in Abenix's Builder, registered in the agent catalogue, and (in the case of the edge agent) marked `edge_compatible: true`.

The first agent is `iot-pump-edge-classifier`. It is an edge agent that runs on the Rust runtime sitting on the platform jack-up. Its job is to take a one-second vibration window and classify it as low, medium, or high severity. To do this it uses two tools: `code_executor`, which it calls with a small Python snippet that computes RMS amplitude and the dominant FFT frequency, and `mqtt_publish`, which it uses to write the verdict back to the plant's alarm topic and (when severity is high) to a cloud escalation topic.

The second agent is `iot-pump-diagnosis`. It is a cloud agent that runs in the cluster. Its job is to take a high-severity window and decide what failure mode is in progress (bearing, imbalance, misalignment, cavitation) and what the next maintenance action should be. It uses three tools: `tsdb_query` to read the last 30 days of vibration RMS for the asset from a TimescaleDB hypertable, `kb_search` against the `industrial-iot-knowledge` collection to find the matching SOP, and `connector_call` to draft a work order in the CMMS.

The flow.

```
vibration.raw      edge agent      severity      diagnosis
  sensor publish  -----> classifier ---> publish  -----> agent
                        (47 ms)      plant/pump/   (cloud,
                        severity      severity      8s)
                        + escalation ---> CMMS
                        topic          work order
                        (high only)
```

A simplified timeline of one event:

```
T+0 ms    sensor publishes vibration window to pump/vibration.raw
T+12 ms   edge runtime forwards to the classifier agent
T+47 ms   classifier returns high; publishes to severity + escalation
T+50 ms   alarm desk light goes amber
T+1.4 s   cloud runtime picks up escalation; wakes diagnosis agent
T+8 s     diagnosis pulls 30 days of RMS, runs kb_search,
           calls SAP PM connector, drafts a work order
T+9 s     work order id published back to the edge
T+9.1 s   edge classifier stores the WO id; suppresses duplicates
```

What the operator gets: sub-second local detection, multi-second cloud diagnosis with full historical context, and an automatically drafted work order without humans touching a keyboard.

What the auditor gets: every step in [/executions](#), the edge run with its bundle digest, the cloud pipeline with all six agent outputs, and the WO with the audit ID embedded in its description.

## Scenario 2: pharma cold-chain excursion

The plant is a pharmaceutical distributor moving vaccines across a long route. The shipment is in a refrigerated container with a Sensitech logger publishing temperature every five minutes over GSM to the MQTT broker.

The problem is that vaccines have to stay between 2 °C and 8 °C. An out-of-band reading needs adjudication: was it a real loss? Was the cooling system at fault? Who pays? The decision is regulated and audit-grade. It cannot live in a spreadsheet.

This scenario uses three agents that have been authored in Abenix's Builder. One is configured for the edge; two run in the cloud.

The first agent is [iot-coldchain-monitor](#). It is an edge agent running on a small Python runtime in the truck's cabin computer. Its job is to watch the rolling 30-minute temperature window and detect when readings go out of band. It uses two tools: [windowed\\_state](#), which keeps the rolling buffer per shipment id, and [mqtt\\_publish](#), which it uses to package the breach (timestamp, readings, GPS) into a packet and post it on a cloud escalation topic.

The second agent is [iot-excursion-adjudicator](#). It is a cloud agent. Its job is to decide whether a partial-loss claim should be filed. It uses three tools: [kb\\_search](#) against the GDP SOP collection to look up regulatory thresholds for the product class, [connector\\_call](#) against the telematics API for cooling-system telemetry around the breach window, and [approval\\_gate](#) to require sign-off from two licensed pharmacists with a TTL.

The third agent is [iot-claims-dispatcher](#). It is a cloud agent. Its job is to file the partial-loss claim with the broker portal once the adjudication is approved. It uses one tool: [connector\\_call](#) against the broker-portal connector.

The flow:

```

reading 9.4 C      edge monitor      cloud adjudicator
sensor publish ----> agent      -----> agent      ---> approval
                    (windowed      (kb_search,      gate
                    state)          connector_call,  (2 sigs)
                                   reasoning)         |
                                                    v
                                                    dispatcher
                                                    files claim
                                                    |
                                                    v
                                                    claim id back
                                                    to the edge

```

Why this could not be a single edge agent: the decision needs the GDP knowledge graph, a multi-step connector call, and two regulated humans. None of that fits on the truck.

Why this could not be a single cloud agent: if the truck is offline, the cloud never sees the reading. The split (edge does detection, cloud does adjudication) makes the system robust to network drops.

### Scenario 3: alarm-desk noise filtering

The plant is a high-speed bottling line that runs around the clock in three shifts. The control system fires a steady stream of alarms; an experienced operator can ignore most of them. New operators get overwhelmed and either stop the line for non-issues or miss the real ones.

The problem is alarm fatigue. Real failures hide in the noise. An auditor's last finding said "operator response is inconsistent across shifts."

This scenario uses three agents that have been authored in Abenix's Builder. One runs at the edge; two run in the cloud.

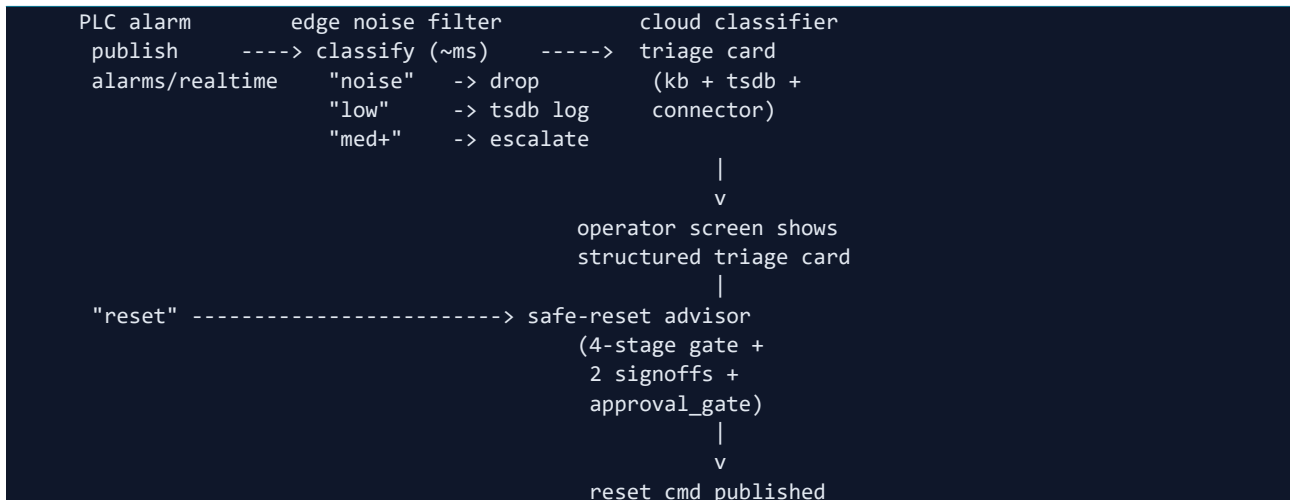
The first agent is [iot-alarm-desk-noise-filter](#). It is an edge agent running on the C runtime on a constrained PLC gateway. Its job is to classify each incoming alarm as noise, low, medium, high, or critical, using a small rule set plus recent context. It uses three tools: [windowed\\_state](#) to remember the last several minutes of alarms for context, [code\\_executor](#) to run a small Python pattern matcher (about 30 lines), and [mqtt\\_publish](#) to forward classified alarms to either a tsdb log topic or a cloud escalation topic depending on severity.

The second agent is [iot-alarm-desk-alarm-classifier](#). It is a cloud agent. Its job is to do deeper reasoning on medium-and-above alarms by cross-referencing line state, the last shift handover notes, and the maintenance log. It uses three tools: [kb\\_search](#) against the alarm-codes KB, [tsdb\\_query](#) against the line-state hypertable, and [connector\\_call](#) to read the maintenance log.

The third agent is [iot-alarm-desk-safe-reset-advisor](#). It is a cloud agent. Its job is to run a four-stage safety gate (hard interlocks, authority matrix, context preconditions, minimum-privilege command) and produce an explicit allow or deny when an operator wants to reset an alarm. It uses two tools: [kb\\_search](#) against the protection-coordination KB, and

[approval\\_gate](#) requiring two operator sign-offs before any reset is published.

The flow:



The win: new operators see the same triage as veterans. Bad resets are blocked by the safety gate, not by tribal knowledge. Every reset has two signatures and a structured rationale that an auditor can replay.

## Scenario 4: field-guide assistant

A field technician boards a turbine to investigate an alert. She has a tablet with spotty connectivity, a phone, and her toolbox. Vendor manuals are PDFs in a SharePoint she cannot reach offline.

She needs the OEM-cited repair procedure for a fault code, the torque values, the safety lockouts, and the parts list. She wants to dictate her closeout into a structured work order, not type it.

This scenario uses three agents that have been authored in Abenix's Builder. All three run in the cloud, with the first one accessible through a small offline-capable shadow on the technician's tablet.

The first agent is [iot-field-guide-troubleshoot-assistant](#). It is a cloud agent. Its job is to take a fault code on a given asset and return an OEM procedure, the relevant safety lockouts, a torque table, and a list of similar past work orders. It uses three tools: [kb\\_search](#) against the OEM-manual collection, [image\\_analyzer](#) to read damage photos the technician uploads, and [connector\\_call](#) to query the CMMS for past WOs on the same asset.

The second agent is [iot-field-guide-closeout-documenter](#). It is a cloud agent. Its job is to convert a technician's free-text or voice closeout into a structured work order. It uses two tools: [audio\\_stt](#) to transcribe the audio into text, and [connector\\_call](#) to write the WO back to the CMMS with the structured fields filled in.

The third agent is [iot-field-guide-schedule-optimizer](#). It is a cloud agent. Its job is to propose a seven-day technician schedule given the day's backlog, technician availability, weather forecast, and ferry schedule. It uses two tools: [code\\_executor](#) (which it invokes with a Python snippet that calls Google OR-tools), and [connector\\_call](#) to read availability from HRIS, the weather feed, and the ferry portal.

The flow:

```
Tech opens app, types "E-1428"
  v
troubleshoot agent runs
- kb_search OEM manual
- kb_search safety procedure
- connector_call past WOs
- synthesises procedure + safety + parts + similar
  v
Tech reads, performs repair, taps mic:
"replaced oil cooler element, torque 48 Nm,
ran 30-min recirculation, no leaks, all clear"
  v
closeout documenter runs
- audio_stt -> text
- structured extraction
- connector_call -> WO closed
  v
Done.
```

The edge angle: when her connection drops mid-procedure, a small Python edge runtime on her tablet keeps a recently cached version of the procedure and accepts her voice closeout. When the link comes back, the closeout flows up and the WO is updated.

## Scenario 5: substation safe-reset

The plant is a medium-voltage distribution substation. A protection relay has tripped on a phase imbalance and the operator at the control room wants to reset and re-energise. There are many substations on this utility's network and the control room has a small operator team.

The problem is that resetting a relay without confirming the upstream cause is exactly how transformers catch fire. The SOP is several pages long and different operators apply it differently.

This scenario uses two agents that have been authored in Abenix's Builder. One runs at the edge; one runs in the cloud.

The first agent is [iot-substation-anomaly-classifier](#). It is an edge agent running on the Rust runtime inside the substation cabinet. Its job is to watch relay events and classify them as transient, persistent, or fault. It uses two tools: [windowed\\_state](#) to keep a short history of recent events for the same feeder, and [mqtt\\_publish](#) to forward fault and persistent events to a cloud escalation topic.

The second agent is [iot-substation-safe-reset-advisor](#). It is a cloud agent. Its job is to run the same four-stage safety gate as the alarm-desk advisor, but with substation-specific rules. It uses three tools: [kb\\_search](#) over the utility's protection-coordination KB, [tsdb\\_query](#) over the upstream load history, and [approval\\_gate](#) requiring sign-off from a senior operator and a relay-protection engineer.

The flow at a high level: the edge classifier sees a trip and publishes the event to a cloud escalation topic. The cloud agent runs the four-stage gate. Two-signoff approval goes to the senior op and the protection engineer. On approval, the cloud agent publishes a reset command on [substation/sub-7/reset-cmd](#), the edge subscriber forwards it to the relay, the relay re-energises. The total wall clock is mostly the human signoff.

The auditor gets the same [/executions](#) page as everything else: every reset has the four gate stages, the two signatures, the cited KB clauses, and the upstream load chart. Replayable.

---

# Part VI — How an application uses Abenix

---

The first two triggers we covered are sensors and the platform UI. The third trigger is another application: your software calling Abenix as a service.

## The SDK pattern

Abenix ships three SDKs: Python, TypeScript, and Java. Same wire format and same mental model.

```
# Python
from abenix_sdk import Abenix, ActingSubject
forge = Abenix(
    base_url="https://abenix.your-corp.com",
    api_key="af...", # platform-issued
    act_as=ActingSubject(
        "contractiq", user_id, email, name # end-user identity
    ),
)
result = forge.execute(
    "contractiq-extract-clauses",
    {"contract_id": cid, "policy_kb": "msa-2026"},
)
print(result.output)
```

```
// Java
try (Abenix forge = Abenix.builder()
    .baseUrl(System.getenv("ABENIX_API_URL"))
    .apiKey(System.getenv("CONTRACTIQ_ABENIX_API_KEY"))
    .actAs(new ActingSubject("contractiq", userId, email, name))
    .build()) {
    ExecutionResult res = forge.execute(
        "contractiq-extract-clauses",
        Map.of("contract_id", cid, "policy_kb", "msa-2026"));
    System.out.println(res.output());
}
```

Three things to notice.

First, the application holds one platform API key. A SaaS app does not mint a key per end-user.

Second, the application says who the work is for. The `actAs` parameter passes the end-user's identity through to Abenix on every request. Internally, the platform routes that identity into quotas (the end-user's monthly cap, not the application's), the audit log (the row says "user alice@bigco.com ran X at 14:32 UTC", not "the contractiq service ran it"), tenant isolation (the end-user's own tenant scope is what reads/writes happen against), and RBAC (the end-user's role and permissions are enforced).

This is what makes Abenix a viable backend for a SaaS product. Five showcase apps in this repository use this pattern.

Third, the API surface is narrow. `forge.execute(slug, input)` is essentially a function call. The agent might be one model call or a 9-step pipeline; might run in the cloud or proxy down to the edge. The application does not need to care.

## Sync, streaming, and async

| Mode      | When to use                                                                                                                                                             |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sync      | Quick agents (under a minute).<br><code>forge.execute(slug, input)</code> blocks until done.                                                                            |
| Streaming | Long agents where the user wants progress.<br><code>forge.stream(slug, input)</code> returns an async iterator over Server-Sent Events.                                 |
| Async     | Fire-and-forget. <code>forge.execute(slug, input, wait=False)</code> returns an <code>execution_id</code> . Poll <code>/executions/{id}</code> or subscribe to its SSE. |

ClaimsIQ uses streaming for its live-DAG view. ContractIQ uses sync for clause extraction. The IoT live-mode toggle uses async because the run is a multi-minute pipeline.

## Idempotency

If your application retries on failure (cron, queue worker, third-party webhook), pass an `Idempotency-Key`:

```
result = forge.execute(slug, input, idempotency_key=f"shipment-{shipment_id}")
```

For 24 hours after the first call, any subsequent call with the same key returns the same cached result without re-running. Useful for "the network blipped, did my filing actually go through" cases.

## Subscribing to outputs

For event-driven applications, you do not always want to call Abenix. You want Abenix to call you. Two patterns.

A webhook is registered on the agent's manifest. The platform posts the result to your URL when the execution completes.

An MQTT subscription is on `executions/{slug}/output`. The platform publishes every completed execution. The cold-chain claims dispatcher works this way.

---

# Part VII — Anatomy of one execution

---

The agent we use as the example is `iot-pump-edge-classifier`. It has been authored in Abenix's Builder, marked `edge_compatible: true`, and deployed to a gateway through the lifecycle described in Part IV. Its job is to take a one-second vibration window and return a severity verdict. It has access to two tools: `code_executor` (which it uses to compute RMS and the dominant FFT frequency from the samples) and `mqtt_publish` (which it uses to write the verdict back to the plant's alarm topic).

What follows walks line-by-line through one execution of this agent so you can see what actually happens inside the runtime. You do not need to read this code yourself; it is here to show that the runtime is not magic.

## Steps 1 to 3: receive input, load manifest, build the LLM call

```
# 1. Caller sends a window
POST /agents/iot-pump-edge-classifier/execute
Body: {"samples": [0.012, -0.014, ...], "sample_rate_hz": 4096}
# 2. The runtime loads the manifest
slug = "iot-pump-edge-classifier"
manifest = yaml.safe_load(open(f"/var/edge/agents/{slug}/agent.yaml"))
system_prompt = open(f"/var/edge/agents/{slug}/system_prompt.md").read()
# 3. Construct the LLM call
messages = [{"role": "user", "content": json.dumps(input_body)}]
tools = [
    {"name": "code_executor", "description": "run a python snippet"},
    {"name": "mqtt_publish", "description": "publish to a topic"},
]
```

## Steps 4 to 6: first model call, code\_executor tool call

```

# 4. The agent loop, first iteration
response = anthropic.messages.create(
    model=manifest["model"],
    system=system_prompt,
    messages=messages,
    tools=tools,
    max_tokens=manifest["max_tokens"],
)
# 5. The model wants a tool: code_executor
# response.content[0] = {
#     "type": "tool_use",
#     "name": "code_executor",
#     "input": {"code": "import numpy as np; ..."}
# }
# 6. Runtime executes the snippet in a sandboxed subprocess
result = subprocess.run(
    ["python3", "-c", response.content[0].input["code"]],
    capture_output=True,
    timeout=5,
)
tool_output = result.stdout.decode()
# tool_output = '{"rms": 0.61, "peak_hz": 102.3}'

```

**Steps 7 to 11: feed back, mqtt\_publish, return final result**

```

# 7. Feed the tool output back to the model
messages.append({"role": "assistant", "content": response.content})
messages.append({
    "role": "user",
    "content": [{
        "type": "tool_result",
        "tool_use_id": response.content[0].id,
        "content": tool_output,
    }],
})
response = anthropic.messages.create(...same args...)
# 8. The model now wants mqtt_publish on a permitted topic
# response.content[0] = {
#     "type": "tool_use",
#     "name": "mqtt_publish",
#     "input": {"topic": "plant-3/pump/p-12/severity",
#               "payload": '{"severity":"high",...}' }
# }
# 9. Runtime checks the topic against the agent's MQTT publish ACL
allowed = manifest["edge_constraints"]["mqtt_publish"]
# allowed = ["plant-3/pump/+severity"]
# matches the requested topic, so permitted
mqtt_client.publish(
    "plant-3/pump/p-12/severity",
    response.content[0].input["payload"],
    qos=1,
)
# 10. Feed back, model returns final text response
response = anthropic.messages.create(...same args...)
# response.content[0].text = '{"severity":"high","peak_hz":102.3,"rms_g":0.61}'
# 11. Runtime returns the result to the caller
return {
    "slug": slug,
    "duration_ms": int((time.time() - t0) * 1000),
    "result": json.loads(response.content[0].text),
}

```

The agent loop is short. The interesting work happens inside the model and inside the tools. The runtime is glue.

---

# Part VIII — Security and governance

---

The edge story rests on three security primitives. They are worth understanding even if you are not the IT/OT lead.

## Bundle signing

Every `.agent` bundle is signed by the platform with RSA-PSS. In plain terms: the platform has a private key that only it holds. It uses the key to sign the bundle's contents. The signature goes inside the bundle. The platform's public key is mounted into every gateway runtime at install time.

The runtime refuses to load a bundle whose signature does not verify. There is no override flag. An attacker who steals a bundle off the wire and modifies it cannot get it loaded, because the modification breaks the signature. An attacker who steals an unmodified bundle and ships it to a different gateway can run it on that gateway, but only if they also stole that gateway's `af_` token. That is the threat model.

## The tool whitelist

Edge agents can only use tools from this short list:

```
mqtt_publish, mqtt_subscribe, current_time,
windowed_state, connector_call, code_executor
```

Specifically forbidden: `knowledge_search`, `kb_query`, `atlas_*`, `mcp_*`, `agent_step`, `approval_gate`, `human_approval`, `pipeline_*`. These either need cluster-only state, require a human signoff, or call back to the platform. They have no business on a gateway.

The whitelist is enforced twice: once by the compiler (refuses to build a bundle that violates it) and once by the runtime (refuses to load even a correctly signed bundle that violates it).

## MQTT ACLs per agent

Even within the allowed tools, agents are constrained by topic ACLs. The agent's manifest declares what it is allowed to publish to and subscribe to:

```
edge_constraints:
  mqtt_publish:
    - plant-3/pump/+/severity
    - cloud/escalation/pump-anomaly
  mqtt_subscribe:
    - pump/vibration.raw
```

The runtime enforces these at every `mqtt_publish` call. If a compromised agent (say, the model gets jailbroken) tries to publish to a topic outside its ACL, the publish is dropped and logged. This is defence in depth against prompt injection.

## Audit trail

Every execution, cloud or edge, emits a structured log line that flows into the platform's audit trail. The line includes the agent slug, the bundle digest, the gateway id (if edge), the caller identity (via `actAs`), the tool calls made, the topics published to, the duration, and the failure code if any. The trail is tenant-scoped and integrity-hashed.

This means every reset of every relay, every claim filed, and every work order opened has a full chain of custody from "human request" through "agent reasoning" to "physical action". Replayable in `/executions`. Searchable by failure code, tenant, or gateway.

## Authentication

| Channel                  | Auth                                                                                                                 |
|--------------------------|----------------------------------------------------------------------------------------------------------------------|
| Web UI to API            | JWT (RS256), refresh-token flow                                                                                      |
| SDK (application) to API | API key ( <code>af_</code> ), SHA-256-hashed at rest, scoped permissions, optional <code>actAs</code> for delegation |
| Edge runtime to API      | Same <code>af_</code> API key, sent as <code>Authorization: Bearer af_...</code>                                     |
| MQTT broker              | Configurable. Anonymous in dev. Username + password or mTLS in production.                                           |
| Bundle integrity         | RSA-PSS signature on the bundle, independent of MQTT auth                                                            |

A compromised broker still cannot ship malicious bundles, because the runtime checks the signature.

---

# FAQ

---

## **Do I need MQTT in my plant to use Abenix?**

No, but you will get more out of the platform if you have one. The cloud half works without MQTT. The edge half requires MQTT for OTA bundle delivery and async invocation. You can ship the same mosquito we ship in our helm chart; it boots in 30 seconds and runs in 30 MB.

## **Can I run an edge agent without the cloud?**

For short windows, yes. A deployed bundle keeps running until the next OTA push. The runtime queues telemetry and re-syncs when the link returns. For days-or-longer offline operation, the longer-term plan is to bundle a distilled small model so the edge does not need to call out to a cloud LLM at all. Today the edge calls out to the configured LLM provider over HTTPS, through whatever uplink is available.

## **How big is the edge runtime, really?**

Python is around 80 MB, Rust around 25 MB, C around 12 MB. The C variant fits comfortably on small ARM gateways with 256 MB of RAM.

## **Can two agents run on the same gateway?**

Yes. A gateway hosts as many agents as memory allows. Each one gets its own subdirectory under `/var/edge/agents/<slug>/` and its own slug-named topic subscription.

## **What if I need a tool that is not on the edge whitelist?**

Split the agent. Have the edge agent do the local-decision part and escalate to a cloud agent that has the missing tool. The cooperation pattern in Part IV covers this.

## **How do I update an agent that is already deployed to many gateways?**

Click Deploy with the new revision. The platform fans out one MQTT publish per gateway-deploy-topic. Each runtime hot-loads the new bundle.

## **How do I roll back?**

Each bundle has a digest, and the gateway keeps the last few versions on disk. The platform UI has a Rollback button per gateway-agent pair that re-publishes the prior bundle.

## **How does this compare to AWS Greengrass or Azure IoT Edge?**

Greengrass and IoT Edge are device-fleet-management platforms. They ship Lambda functions or container images. Abenix is an AI-agent platform: it ships signed agent

definitions that include the prompt, the model, and the tool wiring. You can run Abenix's edge runtime on top of Greengrass or IoT Edge if you want their device management, the same way you can run our Helm chart on AKS or EKS.

### **Is the edge runtime open source?**

Yes. MIT licensed, same as the platform. Source for all three variants is in this repository under [apps/edge-runtime](#), [apps/edge-runtime-rust](#), and [apps/edge-runtime-c](#).

### **How do I monitor the fleet?**

The [/edge](#) page lists every registered gateway with its [last\\_seen\\_at](#) and the digests of deployed bundles. Each runtime exposes Prometheus metrics that get scraped into the platform's [stack](#). Grafana dashboards ship in [infra/helm/abenix/templates/grafana-dashboards/](#).

---

# Glossary

---

**Agent.** A small program with a name, a prompt, a model, and a list of tools. The unit of deployment in Abenix.

**Tool.** A named function the agent can call, such as `mqtt_publish`, `kb_search`, or `code_executor`. About 100 tools ship in the box.

**Pipeline.** A DAG of agents and tools. Switch nodes branch on output, loop nodes iterate.

**Bundle (.agent).** A signed tarball containing one agent's manifest, prompt, and inline tools. The OTA payload to the edge.

**Gateway.** A runtime pod sitting on the plant LAN, hosting deployed bundles. Python, Rust, and C variants ship.

**Edge constraint.** A per-agent declaration of max payload size, max runtime, and allowed MQTT topics. Enforced at compile and at load.

**Topic.** An MQTT address; a forward-slash path like `pump/vibration.raw`.

**QoS.** MQTT delivery guarantee: 0 (fire-and-forget), 1 (at least once), 2 (exactly once).

**Retained message.** An MQTT message the broker keeps as the current state for late subscribers.

**actAs.** The Abenix delegation pattern. An application holds one platform key and passes per-end-user identity on each call.

**Idempotency-Key.** An HTTP header that makes a retried `/execute` return the cached previous result for 24 hours.

**Approval gate.** A pipeline node that blocks execution until N humans sign off, with a TTL.

**Failure code.** A stable string identifier for a class of failure (`LLM_RATE_LIMIT`, `SANDBOX_TIMEOUT`, and so on). The `/alerts` page groups by it.

**Atlas.** The unified ontology and KB canvas. One graph for the document corpus and the typed concepts.

**DLQ.** Dead-letter queue. Failed executions land here with one-click replay or discard.